TransRepair: Context-aware Program Repair for Compilation Errors

Xueyang Li*
SKLOIS, IIE, CAS
School of Cybersecurity, UCAS
China

Shangqing Liu*
Nanyang Technological University
Singapore

Ruitao Feng
University of New South Wales
Australia

Guozhu Meng†
SKLOIS, IIE, CAS
School of Cybersecurity, UCAS
China

Xiaofei Xie
Singapore Management University
Singapore

Kai Chen
SKLOIS, IIE, CAS
School of Cybersecurity, UCAS
BAAI
China

Yang Liu
Nanyang Technological University
Singapore

ABSTRACT

Automatically fixing compilation errors can greatly raise the productivity of software development, by guiding the novice or AI programmers to write and debug code. Recently, learning-based program repair has gained extensive attention and became the state-of-the-art in practice. But it still leaves plenty of space for improvement. In this paper, we propose an end-to-end solution *TransRepair* to locate the error lines and create the correct substitute for a C program simultaneously. Superior to the counterpart, our approach takes into account the context of erroneous code and diagnostic compilation feedback. Then we devise a Transformer-based neural network to learn the ways of repair from the erroneous code as well as its context and the diagnostic feedback. To increase the effectiveness of *TransRepair*, we summarize 5 types and 74 fine-grained sub-types of compilations errors from two real-world program datasets and the Internet. Then a program corruption technique is developed to synthesize a large dataset with 1,821,275 erroneous C programs. Through the extensive experiments, we demonstrate that *TransRepair* outperforms the state-of-the-art in both single repair accuracy and full repair accuracy. Further analysis sheds light on the strengths and weaknesses in the contemporary solutions for future improvement.

CCS CONCEPTS

• **Software and its engineering → Software defect analysis**; **Automatic programming**; • **Computing methodologies → Machine translation**.

*Both authors contributed equally to this research.
†Corresponding author.

ASE '22, October 10–14, 2022, Rochester, MI, USA

KEYWORDS

Program repair, compilation error, deep learning, context-aware

1 INTRODUCTION

Automated program repair, which aims at fixing the underlying errors in a program, plays a critical role in the software development cycle. Generally, it can be roughly categorized into program logical error fixing and compilation error fixing. Compared with the widespread attention on repairing program logical errors [8, 22, 29, 37], the compilation error fixing has just gotten into the horizon of researchers in the past few years [2, 19, 50]. Besides raising the productivity of software development, it can also facilitate the AI programming, such as code generation [7, 12] and binary decompilation [16, 24]. Recent research shows that AI programmers may produce lots of erroneous code (including compilation errors) as human novice programmers did [45]. However, it is non-trivial yet to automatically fix compilation errors in an undocumented program [13]. Moreover, the error messages returned by a compiler may be obscure and cryptic considering the compiler is evolving with new features and optimization techniques [44]. As a consequence, it is desired and beneficial that the program with compilation errors can be automatically repaired to raise programming productivity and prompt AI programming.

Automated program repair for compilation errors is a far-from-settled problem. Prior studies [2, 6, 19, 39] directly utilized RNN-based encoder-decoder framework to take as input the broken program to generate the exact fix. However, the selected model architecture has the limited learning capacity and drawbacks such as RNNs struggle with long-range dependencies in a sequence. Furthermore, other studies [1, 38, 50] have demonstrated that the compiler diagnostic feedback is valuable to improve the accuracy.

```
Broken Code:
    #include<stdio.h>
    #include<stdlib.h>
    int N;
    int main()
    {
        int n,i;
        scanf("%d", &n);
        int A;
        N=n;
        A=(int *)malloc(n*sizeof(int));
        for(i=0;i<n;i++) scanf("%d ", &A[i]);
    }
GCC Feedback: line 12 Error Message: subscripted value is
↪ neither array nor pointer nor vector
```

Figure 1: The broken code with its compiler message.

For example, DrRepair [50] proposed to construct the program-feedback graph by connecting same identifiers in source code and symbols (*e.g.*, identifiers, types, operators) in the compiler feedback to encode the semantic correspondence and further utilized graph attention network to capture relations between program and message to fix the broken program. DrRepair has achieved the state-of-the-art performance and outperforms previous approaches that ignore the compiler feedback significantly. However, through our in-depth analysis of the feedback produced by the compiler, we find that the correspondence between the location of the broken code and the error message is not completely accurate. A simple example is illustrated in Figure 1. It shows that the feedback produced by GCC compiler consists of the reported line number (*i.e.*, line 12 in Figure 1) and the error messages. The root cause is at line 9 and the identifier *A* should be declared as a pointer type (*i.e.*, "int A" → "int *A"). However, the feedback produced by GCC depicts that there is an error at line 12. The location of the root cause in the broken program and the line number produced in the feedback are mismatched, which demonstrates that the error message fails to reveal the reason of this error. Hence, the graph constructed based on the feedback may not capture the essence of errors. Furthermore, in Figure 1, we also find that there is no symbol existing in the feedback and the program-feedback graph cannot be constructed. Finally, the context (highlighted in blue of Figure 1) can infer that the identifier A is a pointer rather than an integer, but this part of context information is ignored in current works.

On the other hand, high quality training data is demanding for learning-based program repair [43]. There are two open-source datasets with compilation errors of C programming language (*i.e.*, DeepFix [19] and TRACER [2]). The DeepFix dataset contains 37,415 correct programs and 6,971 broken programs, which fail to pass the compilation and TRACER contains 21,994 single-line error programs[1]. Although the dataset is further augmented [50] by a program corruption approach, the synthesized code is limited in error types so that the repair performance will be greatly degraded in front of arbitrary errors in reality. Additionally, the data for training a repair model is not yet extensively evaluated, so it is unclear what types of errors cannot be well learned and the underlying cause.

To address the aforementioned challenges, in this study, we propose a context-aware program repair technique to fix compilation errors. To enrich the diversity of the broken programs, we conduct a comprehensive analysis on compilation errors from two real-world programs (*i.e.*, DeepFix and TRACER) and relevant questions in StackOverflow. We summarize these common compilation errors and obtain **74** compilation errors in terms of syntax and semantics. We further classify these errors in **5** different groups. We propose fine-grained perturbation strategies for each type of tokens in a program, and develop an automated approach to break programs with specific errors. In such a manner, we synthesize a dataset with **1,821,275** broken programs in line with the real error scenario. We further devise a Transformer-based program repair model (*i.e.*, *TransRepair*) that takes as input each line of a broken program, the context for each line of statements and the error message to locate the errors and then fix them. A pointer mechanism is incorporated into the model that proves to be effective in solving errors involved with *out-of-vocabulary* code tokens. The extensive experiments on two open-source dataset DeepFix and TRACER have demonstrated that *TransRepair* outperforms current state-of-the-art DrRepair in repair accuracy by 4.66% and 5.7% on DeepFix and TRACER, respectively. The ablation studies for both model components and training data reveal the importance in lifting the repair efficacy. The result analysis concludes that our approach performs the best in fixing "statement" errors and gains more advantages for "type mismatch" and "variable declaration" errors compared to DrRepair.

Contributions. We summarize the main contributions as follows:

- We empirically analyze the common compilation errors from two public datasets and StackOverflow, concluding 74 concrete patterns of compilation errors and 5 categories. Based on that, we further design a number of fine-grained perturbation strategies to create a dataset of diverse broken problems.
- We propose a Transformer-based repair model, which takes each line of a broken program, its context and error messages as input to locate and repair the erroneous code. According to the best of our knowledge, we are the first to consider the context information for repairing the compilation errors.
- The extensive experiments on two open-source datasets demonstrate that *TransRepair* outperforms the state-of-the-art in both single repair and full repair. Moreover, the ablation and failure case studies identify the inherent advantages and limits in light of different types of errors.

More details about code, model and experimental results can be accessed from [28] to benefit the academia and industry. The rest of this paper is organized as follows. Section 2 presents an overview of our approach. Section 3 introduces the data synthesis to construct a corrupted dataset. Section 4 and Section 5 are the detailed presentation of data parsing and model design. We introduce the experimental setup and analyze experimental results in Section 6 and Section 7 respectively. Section 8 details the threats to validity of our work, followed by the related work in Section 9. We conclude our paper in Section 10.

2 SYSTEM OVERVIEW

In this section, we first formulate the research problem, then provide an overview of our approach.

[1]The exact number is mismatched with the reported number in the original paper [2], since we filter out some obvious error samples.

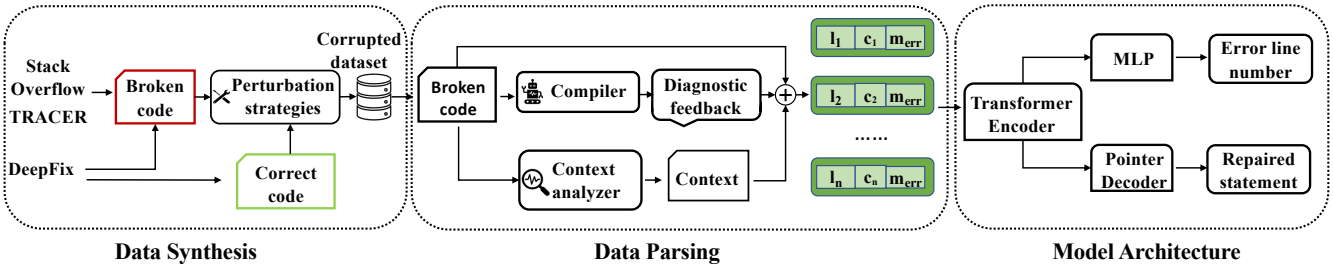


Figure 2: The overview of *TransRepair*

2.1 Problem Formulation

Following the existing works [1, 38, 50], *TransRepair* aims at repairing the program compilation errors by learning the program semantics through deep learning techniques. Formally, given a broken program p from a dataset D (*i.e.*, $p \in D$), where $p = (l_1, l_2, ..., l_n)$, n is the total number of lines in p. Its diagnostic feedback provided by a compiler is defined as a list of $(i_{\text{err}}, m_{\text{err}})$,where i_{err} is the reported line number, and m_{err} is the error message. Since the line number in the diagnostic feedback may not match the line of the root cause in a broken program (shown in Figure 1), the goal of *TransRepair* is to learn a function f from the dataset D that takes $(p, i_{\text{err}}, m_{\text{err}})$ as input and identifies the location k of the erroneous code l_k where $k \in \{1, ..., n\}$, and a repaired version of this statement (*i.e.*, l'_k). The formulation can be expressed as $l'_k = f(p, i_{\text{err}}, m_{\text{err}})$.

2.2 Approach Overview

Figure 2 presents the overview of our approach and it consists of three sequential modules–*data synthesis*, *data parsing* and *model architecture*. In the data synthesis, we first empirically summarize the common compilation errors from multiple error sources including DeepFix, TRACER and a self-curated dataset from StackOverflow. We further design a set of perturbation strategies based on the summarized compilation errors to corrupt the correct programs from DeepFix and construct a new high-quality dataset D that is in line with the real scenario. For each broken program p in the constructed dataset, we compile it to obtain the diagnostic feedback (*i.e.*, $(i_{\text{err}}, m_{\text{err}})$) provided by the compiler. Furthermore, we design a context analyzer to extract the context of each line of code to facilitate learning the context by the model. We take each line l_i, its context c_i as well as the diagnostic feedback $(i_{\text{err}}, m_{\text{err}})$ as the input of the Transformer encoder to learn vector representations. We further apply a fully-connected feedforward network (MLP) to locate the line with error, and a pointer-based Transformer decoder to generate a repair for the error code.

3 DATA SYNTHESIS

In this section, we introduce our data synthesis module that aims at corrupting the correct program by the summarized perturbation strategies to construct a high-quality corrupted dataset in line with the real scenario.

3.1 Taxonomy of Compilation Errors

High quality data (*e.g.*, large number, good diversity and accurate error triage) makes a model better learn the repair rules. The study [50] summarizes common compilation errors for Java, C and C++ programming languages from DeepDelta [38], DeepFix [19] and SPoC [27] respectively. Then five types of errors are specified as well as the corresponding corruption rules for broken code synthesis. However, as we observe, there are more types of compilation errors that appear in reality but not in the their datasets.

In this study, we construct our own dataset by manually analyzing 6,971 erroneous programs in DeepFix and 21,994 programs in TRACER. Furthermore, we conduct an intensive search in StackOverflow to include more diverse errors. Specifically, to obtain a collection of compilation errors, we retrieve the data on StackOverflow with the keywords "[syntax-error] [c]" or "[compile-error] [c]" and get 200 questions ranked by "Highest score" [2]. All the programs as well as their error messages in StackOverflow are enclosed into our dataset.

Manual analysis. We recruited four experts, all of whom have more than five years of programming experience, to analyze the collected program errors from DeepFix, TRACER and StackOverflow. First, we normalize the error messages by removing the specific information such as identifier name and line number, and group them with the same normalized messages into distinct clusters. Then, we spend about six man months to identify the type of errors, and whether an error message is accurate, for example, in revealing the causes of code errors. Specifically, we divide these clusters into four analysis tasks and assign one expert with two of them. Every error message is analyzed by two experts for cross validation. If a disagreement occurs, a third expert will be involved to make the final decision.

The compiler usually conducts the syntax analysis and semantic analysis to ensure the correction of a program. For example, the mistakenly spell of reserved words can incur a syntax error and using a variable without declaration produces a semantic error. As aforementioned, we manually analyze the collected erroneous programs and distill a list of 74 error patterns in total. As shown in Table 1, we further cluster these patterns into five categories within the syntax and semantic analysis phases. This taxonomy is built mainly based on the principles of compiler [3] and the analysis objects in each phase. In particular, a compiler will check whether the program complies with the context-free grammar of C in syntax analysis and produce syntax errors if failed. As observed in the dataset, there are two types of errors-structure error and statement error, significantly varying in influence scope and repair strategies. Structure error defines the misuse or absence of delimiter(s) (*e.g.*, "{", "}", ";") in a statement or a block. It may propagate the influence to the entire program when a brace, for example, is missing. On the

[2]The queried results are as of April, 2022.

Table 1: The analysis of common compiler errors from DeepFix, TRACER and StackOverflow as well as the correspond program perturbation operation, which consists of the operand to change and operations.

Error	Type	Statistics				Operand	Operation		
		DeepFix	StackO.	Trace	Avg.		ADD	DEL	REP
Syntax	structure (struct)	56.51%	14.00%	19.68%	21.28%	punctuator	✓	✓	✓
	statement (stmt)	69.40%	34.00%	69.04%	51.52%	keywords/operator/variable type/name	✓	✓	✓
Semantic	variable declaration (decl)	52.85%	39.50%	20.88%	21.43%	variable type/name	✓		
	type mismatch (tm)	2.95%	4.00%	2.87%	2.17%	variable type/name	✓	✓	
	identifier misuse (im)	2.61%	10.50%	5.47%	3.60%	operator/variable name	✓		✓

contrary, statement errors are caused due to the mistaken tokens in labeled statement, expression statement, selection statement or iteration statement, and the error influence is often confined in a single line. For example, for a correct expression statement "a = a + 1", if "1" is missing, the expression becomes "a = a +", which can definitely cause an error with single-line influence. In semantic analysis, the compiler will build the semantics for the constructs of code as well as their relations in between. Therefore, errors are identified specific to the concrete semantic analysis tasks, such as *scope resolution* and *type checking*. Here we refine semantic errors into three classes, namely "variable declaration", "type mismatch" and "identifier misuse". The "variable declaration" represents the use before the variable is declared. The error of "type mismatch" defines the mismatch of the type or the number of formal parameters of a function. For example, given a function "$f(a, b)$" that allows the invocation with two arguments, however, it is fed with three arguments, *e.g.*, "$f(a, b, c)$", inducing such errors. As for "identifier misuse", for example, a variable is declared as an Integer, so that it cannot be used as a pointer like "int a; a->t=0;".

The statistics of these types of errors in the datasets of DeepFix, StackOverflow and TRACER is also presented in Table 1. As a program may have multiple types of compilation errors, the total ratio of each dataset may exceed 100%. We observe that the distribution of compilation errors are very different across the datasets. Generally, the structure, statement and variable declaration account for the vast majority in the datasets.

3.2 Broken Code Synthesis

To prepare the broken programs with the aforementioned errors, we devise a specific perturbation method to corrupt the correct programs from DeepFix. The code corruption is conducted token-wised, that is, we make changes to a certain code token to produce an error. There are basically three operations in the course of perturbation– *ADD* is to add one token; *DEL* means to remove one token, and; *REP* works as replacing a token with another one. As such, the synthesis of broken code proceeds in the following steps.

Step 1. Given a program, we construct its abstract syntax tree (AST) and identify all the tokens in code, as well as the type of tokens.

Step 2. Configure the corruption procedure by specifying the number of errors made to the code, and the type of errors. Here we create at most five errors for each program, in order to enable the repairer to be able to fix the code with multiple errors.

Step 3. Make the errors specified in the previous step. For each error, we first conduct a global analysis of the target code, select the candidate variable names or symbols for replacement according to the corruption rules, and finally select one of them as the operand. For example, to generate a "statement" error, we can take the keyword, operator, variable type or name in AST as the operand, and perform one of three operations (*i.e.*, add, delete and replace). Table 1 shows the details for perturbation strategies. Noted that, when the operation type is REP, we will first find the tokens in the context based on the specific error type, and then randomly select one from them.

The following part presents how to corrupt programs to generate specific errors.

- **Structure,** which randomly adds, deletes or replaces an punctuator such as ",.;()[]" at the position of punctuator.
- **Statement,** which randomly adds, deletes or replaces a keyword/operator/variable type/variable name at any statement if it has such features.
- **Variable declaration,** which adds a variable type or variable name at the variable declaration/usage statement to corrupt a program.
- **Type mismatch,** which randomly adds or deletes a variable type or variable name in the argument list of the function invocation.
- **Identifier misuse,** which randomly adds or deletes an operator or variable name at the declaration statement.

We present the perturbation strategies with some examples in Figure 3 for better illustration. For each correct program, we repeatedly conduct the code synthesis procedure for 50 times to generate different broken programs and construct a new dataset D. Additionally, compared with [50], our rules for perturbation are summarized from multiple program sources, which are in line with the real-world programming errors. All the above enables us to prepare a better training set for program repair learning. As a consequence, it makes the model to learn more diverse and comprehensive compilation errors, and achieve better repair efficiency as shown in Section 7.1.

4 DATA PARSING

Through Section 3, we can construct a new dataset D, where the program p in this dataset (*i.e.*, $p \in D$) has some compilation errors. In this section, we introduce the module of diagnostic feedback extraction and the context extraction.

4.1 Extraction of Diagnostic Feedback

The previous works [1, 38, 50] have confirmed that the diagnostic feedback could improve the localization and repair accuracy greatly. Hence, we also incorporate it in *TransRepair*. Specifically, since a broken program may consist of multiple errors, making it possible for the compiler to return multiple error messages, we take into

	Correct Code	Broken Code
Structure	{max = cnt;}	{max = cnt;};
	if (temp < a[j]){	if temp < a[j]
Statement	temp = 2 * b	temp = 2 b
	return 0;	0;
	int a, b, c;	float int a, b, c;
Variable Declaration	int array[len]	int array[len] = [0]
	int array[3] = {1, 2, 3}	int array[] = {1, 2, 3}
Type mismatch	mt= maxtill(n)	mt = maxtill()
	s = sum(a, b)	s = sum(a, *b)
Identifier misuse	x = a + i * w;	a + i * w = x;
	scanf("%d", &a);	scanf("%d", &&a);

Figure 3: Examples of synthesized broken code

account all these errors in the training phase. But in the course of the validation phase, we perform an iterative process to repair each error successively. In each iteration, we use the first error which consists of the reported line number i_{err} and the error message m_{err} as [50]. Furthermore, we replace the function name, variable name and self-defined *struct* with the identifier "_<funcN>_", "_<varN>_" and "_<typeN>_" for normalization, where N is the index to denote Nth position. For example, given three variables "a", "b" and "c" in m_{err}, we replace them with "_<var1>_", "_<var2>_" and "_<var3>_" correspondingly.

The processed error message will be fed to the network as a part of the input for the learning module. Normalization can greatly reduce the vocabulary size of the model and has proven to be effective for software vulnerability detection [30, 53]. It is worth mentioning that we retain the names of these identifiers in a mapping table and will recover them after the repair is completed.

4.2 Context Analyzer

As shown in Figure 1, the context (line 11) of the error statement (line 9) could reflect the variable "A" is a pointer rather than an integer. However, existing works [1, 38, 50] usually ignore the context of each statement in learning, which could provide valuable information to program repair. We propose a context analyzer to extract the context (*i.e.*, c_i) of the statement (*i.e.*, l_i) in a broken program (*i.e.*, p) and take it as part of the input for the enhancement.

The extraction procedure is presented in Algorithm 1. Specifically, we define the input as a program text p and a list of dictionaries L, where each dictionary consists of one statement l_i, the empty lists of "vars_declare" and "vars_use" for l_i and a dictionary that stores the context for l_i. The length of the list L is equal to the number of lines for a program p. We first design a lexical analyzer (*i.e.*, function ANALYZER) to take p as input and outputs three sets, which are variable names (var_set), function names (func_set) and type names (type_set) respectively. We analyze the token from the union of these sets to obtain its attribute (declaration or usage) and append it into a list of vars_declare and vars_use from line 2 to line 8. The function IS_DECLARE is designed by analyzing the token. If it is a variable/function name or some types come before it, such as "Integer" or "Float", we believe this token is the declaration and append it into vars_declare. Otherwise we append it to vars_use.

Algorithm 1: Context Analyzer

```
Input: p: program; L: List[
        {
            statement: string;
            vars_declare: [];
            vars_use: [];
            context: {'declare':[], 'use':[]}
        }
];
Output: L
1  var_set, func_set, type_set = ANALYZER(p)
2  foreach line ∈ L do
3      foreach token ∈ var_set ∪ func_set ∪ type_set do
4          if token ∈ line['statement'] then
5              if IS_DECLARE(line, token) then
6                  line['vars_declare'].append(token)
7              else
8                  line['vars_use'].append(token)
9  foreach line ∈ L do
10     line['context']['declare'] = GET_DECLARE_LINES(p,
         line['vars_use'])
11     line['context']['use'] = GET_USE_LINES(p, line['vars_declare'] ∪
         line['vars_use'])
```

Similarly, if the token is a type name and followed by the *"struct"* or *"typedef"*, we also append it to vars_declare. Otherwise, it is appended to vars_use. Once we have the attribute of a token in the statement, we then extract the context. On one hand, for a token in the list of vars_use, we retrieve its nearest declaration statement and construct a list of declaration statements about all tokens from vars_use by the function GET_DECLARE_LINES. On the other hand, for the declared token, we also retrieve its nearest usage statement. Since the declared token is usually introduced by the expression such as "int a = b", where "a" is the declared token and "b" is the usage token, we also retrieve the nearest usage statement for "b" and combine it with the usage statement of "a" to construct a list of usage statements about all tokens from vars_declare by the function GET_USE_LINES. Last, we concatenate the declare context (line[‘context’][‘declare’]) and use context (line[‘context’][‘use’]). We further remove the duplicate and sort them by the order of the original program p, then take it as the context c_i for statement l_i.

5 PROGRAM REPAIR

In this section, we introduce the model architecture of *TransRepair*, which is shown in Figure 4. It is based on the Transformer architecture and consists of three parts: Transformer-based encoder to encode a broken program to obtain the vector representation of each statement; a fully connected forward neural network (MLP) to locate the broken line, and a pointer decoder to generate a correct statement for fixing.

5.1 Encoding Broken Programs

Through Section 4, we obtain the compiler feedback (i_{err}, m_{err}) of the broken program p and the context c_i for each statement $l_i \in p$. To learn the representations, we directly adopt the Transformer

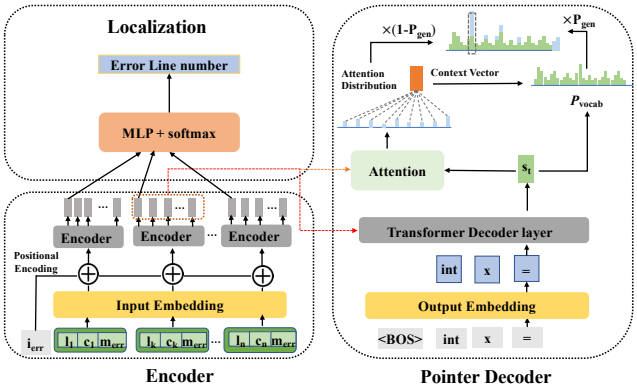


Figure 4: The model architecture of *TransRepair*.

encoder [46] for encoding. Specifically, for each statement l_i with its context c_i and the error message m_{err}, we construct the input s_i in a format of (<BOS>, l_i, <sep>, c_i, <sep>, m_{err}, <EOS>), and feed it to the Transformer encoder to learn the input representation $\boldsymbol{H}_i \in \mathbb{R}^{m \times d}$, where m is the total number of tokens for the input s_i and d is the dimension length. The calculation can be expressed as follows:

$$\boldsymbol{H}_i = \text{Encoder}(s_i) \tag{1}$$

The network architecture of the encoder is almost the same with Vaswani *et al.* [46], which is composed of a stack of N identical layers and each layer has two sub-layers (the multi-head attention layer and the fully connected feed-forward network). The only difference is the positional encoding. We follow DrRepair [50] to add the positional encoding of the line offset with the reported line with error, *i.e.*, $\Delta i = i_{err} - i$, to each token embedding in s_i.

5.2 MLP for Localization

By the Transformer encoder in Section 5.1 for encoding, we obtain each sequence representation $\boldsymbol{H}_i$, where $i \in \{1, 2, \cdots, n\}$ and n is the total lines of a broken program. To locate the line of the error statement (*i.e.*, k), we turn this localization problem into a classification task. Specifically, we extract the vector of $\boldsymbol{H}_i$ at the symbol "<BOS>" (*i.e.*, position "0") as the aggregated sequence vector $\boldsymbol{h}_i$ to represent the sequence s_i, which is similar to CodeBERT [14] and use the softmax function with two fully connected layers to determine whether each statement is erroneous or not according to the predicted probability. The loss function $\mathcal{L}_{\text{loc}}$ can be expressed as follows:

$$\mathcal{L}_{\text{loc}} = -\log \frac{\exp(\boldsymbol{h}_k)}{\sum_{i=1}^{n} \exp(\boldsymbol{h}_i)} \tag{2}$$

where k is the location of the error line in the broken program p and n is the total number of lines of p.

5.3 Pointer Decoder for Fixing

The localization module helps *TransRepair* to locate the error statement in a broken program, we further add a decoder to generate a fixed statement for repair. We adopt the transformer decoder and further add pointer mechanism to copy tokens from the input sequence to overcome the out-of-vocabulary (OOV) issue and improve the accuracy of fixing. Specifically, given the output representation $\boldsymbol{H}_k \in \mathbb{R}^{m \times d}$ of the encoder for the broken statement (<BOS>, l_k, <sep>, c_k, <sep>, m_{err}, <EOS>), where m is the sequence length, at each step t, we utilize the Transformer decoder [46] to receive the word embedding of the previous word and output the hidden states $\boldsymbol{s}_t$. Furthermore, to compute a probability distribution over the input sequence to tell the decoder where to attain to generate the next word, we compute the attention distribution between $\boldsymbol{s}_t \in \mathbb{R}^d$ and $\boldsymbol{H}_k \in \mathbb{R}^{m \times d}$, which can be expressed as follows:

$$\boldsymbol{a}^t = \text{softmax}(\frac{\boldsymbol{H}_k \boldsymbol{s}_t}{\sqrt{d}}) \tag{3}$$

where $\boldsymbol{a}^t \in \mathbb{R}^m$ and d is the dimension length. Then the attention distribution is used to produce a weighted sum of the encoder hidden states (*i.e.*, the context vector):

$$\boldsymbol{h}_t^* = \sum_i \boldsymbol{a}_i^t \boldsymbol{h}_i \tag{4}$$

where $\boldsymbol{h}_i$ denotes i-th vector in $\boldsymbol{H}_k$. The context vector is concatenated with the decoder state $\boldsymbol{s}_t$ and produce the vocabulary distribution P_{vocab}:

$$P_{\text{vocab}} = \text{softmax}(\boldsymbol{V}'(\boldsymbol{V}[\boldsymbol{s}_t; \boldsymbol{h}_t^*] + \boldsymbol{b}) + \boldsymbol{b}') \tag{5}$$

However, Eq 5 could only produce the token from the vocabulary set and the Out-of-vocabulary (OOV) issue, which means that the token is in the input sequence but out of the vocabulary set due to the limited vocabulary length, cannot handle. To address this limitation, similar to See [40], we incorporate the pointer mechanism to allow the network to copy words by pointing and generate words from a fixed vocabulary. Specifically, the generation probability $p_{\text{gen}} \in [0, 1]$ for each step t is calculated from the context vector $\boldsymbol{h}_t^*$, the decoder state $\boldsymbol{s}_t$ and the decoder input $\boldsymbol{x}_t$:

$$p_{\text{gen}} = \sigma(\boldsymbol{w}_{h^*}^T \boldsymbol{h}_t^* + \boldsymbol{w}_s^T \boldsymbol{s}_t + \boldsymbol{w}_x^T \boldsymbol{x}_t + b_{\text{ptr}}) \tag{6}$$

where $\boldsymbol{w}_{h^*}$, $\boldsymbol{w}_s$, $\boldsymbol{w}_x$ and b_{ptr} are learnable parameters and σ is the sigmoid function. p_{gen} is used to choose between generating a token from vocabulary or copying directly from the input sequence. Over an extended vocabulary set, that combining the original vocabulary set with the tokens from the input sequence, the probability distribution is expressed as follows:

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i: w_i = w} \boldsymbol{a_i}^t \tag{7}$$

The loss function for the fixing (*i.e.*, $\mathcal{L}_{\text{gen}}$) can be expressed as follows:

$$\mathcal{L}_{\text{gen}} = -\frac{1}{T} \sum_{t=0}^{T} \log P(w_t^*) \tag{8}$$

where w_t^* is the target word for timestep t and T is the length of the whole sequence. During the training phase, we directly add the loss values of the location model and the fixing model for training:

$$\mathcal{L} = \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{gen}} \tag{9}$$

6 EVALUATION SETUP

In this section, we first introduce the used datasets for different approaches, then briefly introduce the selected state-of-the-art baselines for comparison and the metrics for evaluation. Finally, we present the details about the model configuration of *TransRepair*. We aim at answering the following research questions:

RQ1. What is the performance of *TransRepair* compared with current existing state-of-the-art approaches?
RQ2. Is each component (*i.e.*, diagnostic feedback, context and pointer mechanism) in *TransRepair* effective to improve the repair accuracy?
RQ3. Is each type of the perturbation strategies is beneficial for constructing a more diverse dataset and helping the model improve the performance?
RQ4. When *TransRepair* fails and when it works? An empirical study for investigating the detailed repaired results compared with the state-of-the-art.

6.1 Datasets

In the evaluation, we corrupt the correct programs (in total 37,415) from DeepFix [19] and obtain a total number of 1,821,275 synthetic programs for model training. We conduct a strict deduplication process based on code text similarity[4] to remove the same samples between the training data and testing data. To construct a validation set, we randomly select 2000 samples from TRACER's training set (17,688 in total) for validation. We separately evaluate the performance of the trained model on the testset of DeepFix, which has 6,971 broken programs without ground-truths, and TRACER that contains 3,674 single-line error programs with the provided single-line ground-truths for a comprehensive evaluation. The statistics of the dataset are presented in Table 2. Since the broken programs in the testset of DeepFix may contain errors in multiple lines, we apply *TransRepair* iteratively until the program passes the compilation, or the tries exceed the maximum limit of 5.

6.2 Baselines

DeepFix [19]. DeepFix firstly proposes to adopt the sequence-to-sequence model for fixing programming errors and it concatenates the line number with the line statement as the input for RNNs with the attention mechanism to generate the error line number and the fixed statement. It further designs an iterative strategy to fix multiple errors in a program and the acceptance standard for one line fixing is whether the updated program can yield less error messages than the input program by the compiler. Furthermore, DeepFix also releases a dataset that has been widely used for the evaluation in the follow-up related works for repairing programming errors.
RLAssist [18]. RLAssist proposes a programming language correction framework based on reinforcement learning, which allows an agent to mimic human actions for text navigation and editing. Specifically, by a trained agent, it allows a set of navigation and edit actions to fix a program. The experimental results proved its superiority against Deepfix.
SampleFix [20]. SampleFix proposes a deep generative model to automatically correct programming errors by learning a distribution over potential fixes. A deep conditional variational autoencoder [42] is used to sample the fixes for an erroneous program. Furthermore, a novel regularizer is proposed to encourage the model to generate diverse fixes. The experimental results on the DeepFix dataset have confirmed the effectiveness of the proposed architecture.
MACER [10]. Since the source code of TRACER [2] is not public and we utilize a follow-up work Macer from the same research team, which has confirmed its superiority over TRACER and been made public. Specifically, MACER conducts a code abstraction procedure and formulates this problem as a classification task by predicting the repaired type in a limited repair classes and applies the predicted repairs at the predicted location. Then, it recovers code abstraction and compiles the fixed program for evaluation. The performance on the DeepFix dataset and TRACER dataset confirms the improvement over TRACER.
DrRepair [50]. DrRepair incorporates the diagnostic feedback produced by the compiler for a broken program into a designed model and obtains significant improvements against the previous works. Specifically, DrRepair constructs a program-feedback graph to build the relations between a broken program and the feedback. Then model architecture consists of the bidirectional LSTMs [21] to learn the statement dependencies and the graph attention network [47] to capture the relations between program and feedback. Furthermore, to construct a large scale dataset for pre-training, DrRepair proposes a program corruption procedure to corrupt correct programs from DeepFix. The extensive experimental results on the DeepFix dataset and SPoC dataset prove that DrRepair could achieve the state-of-the-art performance. In our paper, we compare our approach with DrRepair and its alternative without pretrain (*i.e.*, DrRepair w/o pretrain).

For DeepFix, RLAssist and SampleFix, we directly get the reported values in their original papers. For MACER, we utilize the official released model to test the performance on the TRACER testset and DeepFix testset. For DrRepair and *TransRepair*, we separately train the model using the DrRepair-released dataset and our constructed dataset. In addition, in terms of full repair metric on the DeepFix testset, DrRepair sets the beam size to 50 to generate 50 programs for a broken program to test whether this broken program can be fixed. However, by our analysis, we find that the time cost is heavy when setting beam size to 50 and it costs nearly 5 hours for a complete generation process on the DeepFix testset. Considering time and efficiency cost, we set beam size to 5 for DrRepair and *TransRepair* for fair comparison.

6.3 Metrics

We evaluate our approach against other baselines in the metrics of single localize, single repair and full repair accuracy. Since TRACER's testset provides the ground-truths of single-line erroneous program (*i.e.*, each broken program has its correct counterpart), we could use all these metrics for evaluation. However, we only utilize full repair accuracy for the DeepFix testset since it is without ground-truths.
Single Localize. It defines the accuracy of localizing a single error statement in a single-line error program in the TRACER testset.
Single Repair. It is used to evaluate if the generated statement is exactly matched with the ground-truth associated with a broken statement. In this setting, we assume that the error statement is known and we do not need a localization module for localizing an error statement. We use Acc@k to calculate the percentage of the correct results existed in the top-k returned results. Specifically, we adjust the beam search size equal to k to return k results for a broken program and we set k to 1, 5, 10 to evaluate the accuracy of the generated statement in TRACER where each sample has a ground-truth for calculation.
Full Repair. It is designed to evaluate the ability of different approaches on fixing a broken program, which consists of localizing an error statement and further fixing it. Furthermore, it is calculated in the percentage of the generated program that could pass

Table 2: The statistics of the constructed dataset.

Correct Programs	Training set						Validation set	Test set	
	struct	stmt	decl	tm	im	Total		TRACER	DeepFix
37,415	461,663	778,210	261,944	274,366	45,074	1,821,275	2,000	3,674	6,971

Table 3: The experimental results compared with the baselines where the reported values are in percentages and the values with the marker ∗ denote these values are taken from the corresponding papers directly and the marker - denotes the unreported metrics on the specific testset.

Model	TRACER Testset					DeepFix Testset
	Single Localize	Single Repair			Full Repair	Full Repair
		Acc@1	Acc@5	Acc@10		
DeepFix	-	-	-	-	-	27.00*
RLAssist	-	-	-	-	-	26.60*
SampleFix	-	-	-	-	-	45.30*
MACER	31.57	10.34	16.55	38.32	26.08	56.40
DrRepair_ori	84.98	46.24	57.73	60.13	72.66	62.13
DrRepair	**86.72**	48.56	60.23	62.28	77.11	63.87
*TransRepair*_ori	80.19	44.47	58.57	63.39	78.77	66.71
TransRepair	83.21	**49.65**	**61.27**	**65.08**	**82.81**	**68.53**

the compiler in success. We utilize full repair accuracy in both the TRACER testset and DeepFix testset for evaluation. It is noted that the metric "Full Repair" may have limits for evaluation considering the scenarios when the erroneous lines are simply removed rather than correctly edited. It is used here because: 1) the Deepfix dataset has no ground-truths, so we resort to full repair to evaluate the repair performance. Meanwhile, we avoided deleting the entire line which may incur dramatic changes to code semantics. 2) These metrics have also been widely used in [2, 10, 50], with which we can compare with prior studies directly. But we will explore more better metrics in future.

6.4 Model Configuration

TransRepair consists of 5 identical layers for the Transformer encoder and decoder, each layer has 8 heads to learn different subspace features. We select the tokens with the frequency greater than 1 in the training set for constructing our vocabulary set. The word dimension is set to 256 with the positional encoding equals to 50 for the embedding. The optimizer is selected with Adam [26] with an initial learning rate of 0.0001 and batch size of 25. We set the dropout to 0.1 and gradient clipping to 10. All hyper-parameters are tuned on the validation set. The model is trained on a Intel(R) Xeon(R) server with 8 cores, which equips Nvidia 3090 with 24G memory and 2 Nvidia TITAN X with 12G memory and the training process costs around 30 hours.

7 EVALUATION RESULTS

In this sections, we present the experimental results in light of research questions.

7.1 RQ1: Comparisons with Baselines

We compare *TransRepair* with some existing approaches, specifically the row of "{∗}_ori" indicates the model {∗} trained on the original training set that DrRepair released. The experimental results are presented in Table 3.

Among different baselines, we find that DrRepair could achieve the best performance on both TRACER and DeepFix testset, which is in line with the perception that DrRepair is current state-of-the-art approach for repairing program syntax errors. Furthermore, we can observe that *TransRepair* could obtain higher single repair and full repair accuracy than DrRepair when fixing a training set to train (*i.e.*, the original training set that DrRepair uses or our corrupted training set), which illustrates the superiority of our approach in program repair against DrRepair. However, we also find that the accuracy of single localize of *TransRepair* is lower than DrRepair on the TRACER testset, we conjecture that it is caused by the localization requires the exact match to the error line, which is harder for *TransRepair* (Transformer-based) to achieve higher performance compared with DrRepair (LSTM-based). However, the requirement for generating a statement to replace the error statement to pass the compilation is relatively easier for *TransRepair* since the Transformer is more powerful than LSTMs in generating a target sequence even in adverse condition when Transformer has poor ability to accurately localize an error statement. The more powerful generation ability of transformer can be further enhanced by comparing the results of *TransRepair* and DrRepair on the single repair accuracy and this metric is used to evaluate the generated statement is exactly matched with the ground-truth when taking an error statement as the input for the decoder. We can observe that *TransRepair* achieves higher single repair accuracy than DrRepair. Hence, the poor localize accuracy may not significantly impact the repair accuracy in our model and we believe that the metric of repair accuracy plays a critical role for program repair. But we also want to investigate the way to improve our localization accuracy and we leave it as our future work.

In addition, fixing a model (*e.g.*, DrRepair or *TransRepair*), we use our constructed training set or the original training set that DrRepair used for training separately. We could achieve higher repair accuracy on our training set than the original training set that used by DrRepair. It proves that by our designed perturbation strategies, we can construct a training set that is more in line with the real scenario and this dataset could help the model achieve a better performance.

Table 4: The ablation results of *TransRepair*, where w/o denotes the removed component.

Model	TRACER Testset		
	Acc@1	Acc@5	Acc@10
w/o feedback	45.67	58.74	62.38
w/o context	47.09	59.93	64.07
w/o pointer	47.00	60.86	64.78
TransRepair	**49.65**	**61.27**	**65.08**

In summary: *TransRepair* provides higher repair accuracy compared with the state-of-the-art approach DrRepair. We attribute the improvements to the powerful generation ability of the Transformer. Furthermore, by comparing the performance among the training sets that DrRepair used and we constructed, we further confirm that our training set is better for the model to achieve higher performance.

7.2 RQ2: Ablation study of each component in the network architecture

We ablate the performance of *TransRepair* when removing the specific component in the model architecture and maintaining the others for evaluation. The experimental results on TRACER in terms of single repair accuracy are presented in Table 4, where w/o denotes the removed component in *TransRepair* and the model configuration is the same as *TransRepair* for fair comparison.

As shown in Table 4, we can find that the diagnostic feedback plays a critical role in improving the performance and removing it degrades the accuracy significantly. This shows that the diagnostic feedback could supplement some valuable information such as the error line and error message, although in many cases, this information may be inaccurate, it could still contribute the model to achieve higher accuracy when incorporating this part of information. Furthermore, we can observe that the context is also important in improving the performance. Since the context of a statement could reduce the difficulty for the model to learn this statement semantics (See an example in Figure 1, ignoring the context will limit the repair accuracy. The pointer mechanism could effectively alleviate the out-of-vocabulary issue and without it. The repair accuracy drops from 65.08 to 64.78, which demonstrates that there are some target tokens might be out of vocabulary set. Hence, we incorporate the pointer mechanism into the Transformer decoder can mitigate this issue and further improve the performance. Overall, from Table 4, we can conclude that when combing all of these components, *TransRepair* could achieve the best repair accuracy.
In summary: The diagnostic feedback plays a critical role in improving the repair accuracy, however the contextual information and the pointer mechanism is also beneficial for the improvement and when incorporating all of components, *TransRepair* could achieve the best performance.

7.3 RQ3: Ablation study of perturbation strategies in dataset construction

In Section 3, we design a set of 5 perturbation strategies and sample 1-5 strategies to corrupt a correct program and construct a training set for *TransRepair* to learn. In this RQ, we also investigate the effect of each perturbation strategy in building the training set.

Table 5: The ablation results by removing one type of perturbation strategies to construct the training set for learning.

Model	TRACER Testset		
	Acc@1	Acc@5	Acc@10
w/o struct	48.06	60.51	64.43
w/o stmt	47.23	59.56	62.66
w/o decl	46.57	58.49	62.40
w/o tm	47.88	59.51	63.37
w/o im	47.35	59.73	63.06
TransRepair	**49.65**	**61.27**	**65.08**

Specifically, we remove one type of perturbation strategies and maintain the others to build a new training set where the total number of samples in this training set is equal to the original one. Then we train our model on the newly constructed training set with the same model configuration as the original to compare the performance, and the experimental results are presented in Table 5.

We can see that each perturbation strategy is effective in constructing a more diverse training set. When combing all to build a training set, we could achieve the best performance. Specifically, when the training set is constructed without the structure strategy (*i.e.*, the training set has no samples with the type of structure error) has the lowest drop in repair accuracy compared with other strategies. It depicts that the structure error type has the least contributions in constructing a diverse training set to help the model obtain higher repair accuracy. We infer that it is caused by the difficulty in fixing this type of errors. The defined operations only modify punctuators such as "{", "}" in a correct program and these punctuators have no semantic information for a program compared with other types of corrupted operations in Table 1, which involves modifying the variable names to synthesize other error types. Hence, the model is difficult to learn effective patters for structure errors and removing this type of data in the training set cannot lead the model have a significant impact on the repair accuracy. Furthermore, we can see that removing the data that have the "variable declaration (decl)" errors in the training set, the repair accuracy decreases significantly and it demonstrates that adding samples with this type of error could be beneficial for the model learning. We believe that the improvement is due to the designed context analyzer (see Section 4.2), which could extract the contextual information for these variables and it is significantly beneficial for the model to learn effective repair patterns.
In summary: Each type of perturbation strategies is beneficial in constructing a diverse training set. When combining them together and apply them to corrupt correct programs for building the training set, we could obtain the best repair accuracy.

7.4 RQ4: When *TransRepair* fails and when it works?

We conduct a statistical analysis further to compare the repaired results between *TransRepair* and DrRepair. Both models are trained on our constructed dataset and tested on the TRACER testset to verify model's ability to fix different types of program errors. The statistical results are presented in Figure 5, where the number besides the rectangle is the total number of fixes and the ratio of the number of fixes to the total number of this type errors. More details

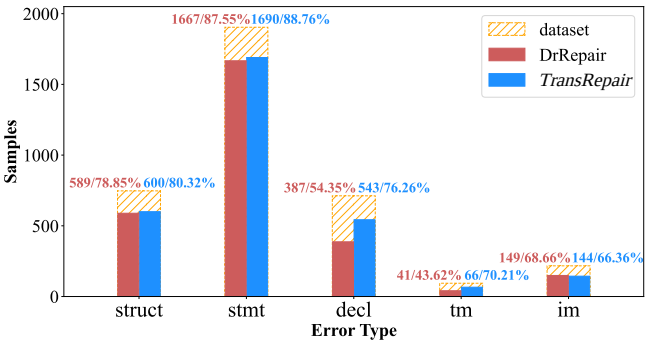


Figure 5: The comparison results for the number of repairs between DrRepair and *TransRepair*.

on the repair efficacy for each concrete error pattern can be found on our website [28].

As illustrated in Figure 5, we find that *TransRepair* is excellent in fixing the errors of "variable declaration (decl)" and "type mismatch (tm)" and has a slight improvement in fixing the errors of "structure (struct)" and "statement (stmt)" while is slightly inferior to fix the error of "identifier misuse (im)" compared with DrRepair. We conjecture that DrRepair could fix more "identifier misuse (im)" errors due to the constructed program feedback graph to capture the variable relations. However, we can also get a competitive performance by the powerful Transformer without the need of the constructed graph. For the other four errors that *TransRepair* could fix better than DrRepair, we attribute the improvement to the used context in helping the model capture the error statement patterns. Especially for the error type of variable declaration, the context information around the error statement is critical to reveal the root cause. Here we present one example with the generated results by *TransRepair* and DrRepair in Figure 6 for better illustration. It shows that the error is due to the variable "n" is not defined at line 5 and its contexts are highlighted in blue at line 3 and line 6. We encode the context (*i.e.*, line 3 and line 6) for this error statement could help *TransRepair* generate a correct statement "for (i = 1 ; i <= N ; i ++) {" for the fixing, while due the lack of the context, DrRepair fails to generate a correct statement to repair this error.

In summary: Generally, *TransRepair* is competitive in fixing the type mismatch error compared with DrRepair, however on other four errors, it could achieve better performance, we attribute the improvement to the utilized context for learning.

8 THREATS TO VALIDITY

Internal validity. One of the threats to validity is the hyper-parameter setting for our approach. We tune our model on the validation set and select the best model based on the repair accuracy and use it for testing. We will explore more hyper-parameters for our approach. Another threat lies in our implementations of the broken code synthesis, context analyzer and model implementation. To reduce this threat, the authors carefully check the correctness of the implementation. We will make our code and the constructed dataset public for further investigation.

External validity. The external threats to validity include the selected datasets, the evaluation of the baselines and the evaluation metrics. In terms of dataset, the training set is constructed from the DeepFix dataset, which is a popular C programming language

```
1  Broken Code:
2      int main ( ) {
3          int N , i , j , k , sum = 0 ;
4          scanf ( "  %d  " , & N ) ;
5          for ( i = 1 ; i <= n ; i ++ ) {
6              for ( j = 1 ; j <= i ; j ++ ) {
7                  if ( k >= 0 )
8                      k = i - j + 1 ;
9                  sum = sum + 1 ;
10                 }
11                 printf ( "  Number of possible triangles is
   ↪  %d, sum ) ;
12                 return 0 ;
13                 }
14     }
15 DrRepair: int N , i , j , k , sum = 0 ;
16 TransRepir: for ( i = 1 ; i <= N ; i ++ ) {
```

Figure 6: The broken program with the generated statement by DrRepair and *TransRepair* for fixing.

dataset for program repairing. We only use three operations (*i.e.*, ADD, DEL and REP) as the building blocks for code mutation. It is intriguing to explore more complex transformation strategies such as multiple operations with logical relations.

Furthermore, we select two testsets (*i.e.*, TRACER and DeepFix) and they are both on C language for evaluation.

We admit that there are some works [38] for other languages like Java but TransRepair cannot be directly used for these languages. We will extend our approach with adaptive language analyzers for other languages in future. Additionally, programs in a more complex system may encounter varying compilation errors considering dependent libraries, templates and generics. Therefore, it may degrade the repair performance of TransRepair, which can be to some extent mitigated by involving more complex programs during the training. Our approach remains effective for AI programming such as automated code completion, we can use our approach to repair the generated programs by AI models. In terms of baselines, for DeepFix, RLAssist and SampleFix, we report the values from the original paper and we believe these reported values are the best for their approaches. For MACER, we also believe the released model is the optimal and for DrRepair, we only adjust the beam search size to 5 and keep the other settings same with the default configuration for reproduction. The default hyper-parameters of DrRepair on our constructed dataset may not be optimal, however on its original dataset that DrRepair uses (the configuration should be optimal), our experiments prove that our approach outperforms it significantly (see Table 3). As for evaluation metrics, we follow DrRepair [50] and utilize the single localize accuracy, single repair accuracy (Acc@1 in our work), full repair accuracy for evaluation. We further add Acc@5 and Acc@10 for a comprehensive evaluation.

9 RELATED WORK

There is a line of works on automated program repair for compilation errors and context-aware program repair. We also briefly introduce some works that use deep learning techniques for different software engineering applications.

9.1 Automated Compilation Error Repair

Over the past years, automated program repairs for compilation errors have attracted widespread attention. DeepFix [19] applied a RNN-based encoder-decoder framework to repair program syntax

errors on C programming language. RLAssist [18] is a follow-up work after the DeepFix. It attempted to use deep reinforcement learning to achieve better repair accuracy. TRACER [2] also adopted RNN-based model to repair the syntax errors and the follow-up work MACER [10] formulated this problem as a classification task. Hajipour *et al.* proposed SampleFix [20], which applied a deep generative model to fix programming errors automatically. These works just utilize the program for the repair, while some external information such as the diagnostic feedback is ignored. To supplement this part of information, SynFix [1] proposed to incorporate the compiler diagnostics from JavaC with the pre-trained RoBERTa for improvement. Yasunaga *et al.* proposed DrRepair [50], which constructed a graph between the diagnostic feedback and the broken program and took them as the input of a self-supervised learning framework to repair the errors. Compared with these works, we craft high-quality training data that is in line with the real scenario and made this well-designed dataset public for further studies. Furthermore, we propose a Transformer-based program repair model with pointer mechanism, which incorporates the broken program and the context and diagnostic feedback to improve repair accuracy.

9.2 Context-Aware Program Program Repair

Because of the complexity of a broken program, it is hard to accurately capture the program semantics. More researchers attempt to utilize the context as the auxiliary information to enhance the fault localization and program repair for logic errors. Specifically, Chilimbi *et al.* [11] proposed a static analysis approach, namely HOLMES, which determines the root causes of targeted bugs based on the run-time profiling information representing program context. Wen *et al.* [48] proposed a context-aware patch generation approach called CapGen, which leverage several novel prioritization methods to enhance the success rate of automatically generated patch for repair. Li *et al.* [29] proposed a context-based code transformation learning approach, namely DLFix, which applied deep learning on automated program repair (APR) without requiring ant hard-coding of bug-fixing patterns. Lutellier *et al.* [37] proposed a combined neural machine translation (NMT) models based context-aware approach, called CoCoNut, which could work on automatic bug repair in multiple programming languages. Kim *et al.* [25] proposed ConFix, which is an context-based automatic patch generation approach for buggy programs. Chen *et al.* [8]proposed a sequence-to-sequence based tool, namely SequenceR, to repair buggy programs by learning from the buggy context of single line repair from human commits. The main difference between it and ours is that we are focusing on compilation errors other than logic bugs. Inspired by above works, in *TransRepair*, we also incorporate context of the error statement for fixing compilation errors.

9.3 Deep Neural Networks for SE Applications

With the rapid development of AI techniques, more researchers attempt to utilize deep learning techniques for software engineering applications. Compared with traditional software analysis techniques, deep learning techniques aim at learning features automatically from a large amount of data. By training a deep neural network and deploying it to the test phase, the superior performance of these models has been confirmed on different applications. For example, Allamanis et al. [5] proposed to construct the program graph and utilized it with Gated Graph Neural Network to learn program semantics for variable misuse detection. Followed by this work, many other works proposed to extract program structures for other applications such as source code vulnerability detection [9, 51], code summarization [15, 33], deep code search [32, 36],neural program decompilation [31]. An empirical study [41] is also conducted to illustrate different program structures to the effect of software engineering applications. Recently, more pre-trained models are proposed to learn general code fragment representation for "code intelligence" such as CodeBERT [14] and GraphCodeBERT [17]. A BART-based pre-trained model CommitBART [35] is also proposed for different commit-related applications such as commit message generation [23, 34], security patch identification [49, 52].

10 CONCLUSION

We develop a Transformer-based approach *TransRepair* to automatically fix compilation errors in C programs. To craft high quality training data, we spend around 2-man months investigating the compilation errors from 28,965 erroneous programs from two public datasets and the Internet, and then distill 74 error patterns that fall into 5 classes. A data synthesis approach is devised by corrupting correct programs into these errors and finally we obtain 1,821,275 erroneous programs of high diversity. *TransRepair* is built on top of Transformer that takes as input the broken program, together with its context and the diagnostic feedback. It integrates the pointer mechanism to address the out-of-vocabulary code tokens and outputs the localization of the error statement and further provides a fixed version. The extensive experiments on two open-source testsets have proved that *TransRepair* outperforms the current state-of-the-art both in repair accuracy.

11 ACKNOWLEDGMENT

We would thank the anonymous reviewers for their valuable comments. IIE authors are supported in part by NSFC (61902395, U1836211), Beijing Natural Science Foundation (No.M22004), the Anhui Department of Science and Technology under Grant 202103a05020009, Youth Innovation Promotion Association CAS, Beijing Academy of Artificial Intelligence (BAAI). This research is also partially supported by the National Research Foundation, Singapore under its the AI Singapore Programme (AISG2-RP-2020-019), the National Research Foundation, Prime Ministers Office, Singapore under its National Cybersecurity R&D Program (Award No. NRF2018NCR-NCR005-0001), NRF Investigatorship NRF-NRFI06-2020-0001, the National Research Foundation through its National Satellite of Excellence in Trustworthy Software Systems (NSOE-TSS) project under the National Cybersecurity R&D (NCR) Grant award no. NRF2018NCR-NSOE003-0001, the Ministry of Education, Singapore under its Academic Research Tier 3 (MOET32020-0004). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

REFERENCES

[1] Toufique Ahmed, Noah Rose Ledesma, and Premkumar Devanbu. 2021. SYNFIX: Automatically Fixing Syntax Errors using Compiler Diagnostics. *arXiv preprint arXiv:2104.14671* (2021).

[2] Umair Z Ahmed, Pawan Kumar, Amey Karkare, Purushottam Kar, and Sumit Gulwani. 2018. Compilation error repair: for the student programs, from the

student programs. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training*. 78–87.

[3] Alfred V. Aho and Jeffrey D. Ullman. 1977. *Principles of Compiler Design* (1 ed.). Addison-Wesley Professional.

[4] Miltiadis Allamanis. 2019. The adverse effects of code duplication in machine learning models of code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153.

[5] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2017. Learning to represent programs with graphs. *arXiv preprint arXiv:1711.00740* (2017).

[6] Sahil Bhatia and Rishabh Singh. 2016. Automated correction for syntax errors in programming assignments using recurrent neural networks. *arXiv preprint arXiv:1603.06129* (2016).

[7] Xinyun Chen, Linyuan Gong, Alvin Cheung, and Dawn Song. 2021. PlotCoder: Hierarchical Decoding for Synthesizing Visualization Code in Programmatic Context. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*. Association for Computational Linguistics, 2169–2181.

[8] Zimin Chen, Steve James Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. SEQUENCER: Sequence-to-Sequence Learning for End-to-End Program Repair. *IEEE Transactions on Software Engineering* (2019), 1–1. https://doi.org/10.1109/TSE.2019.2940179

[9] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 3 (2021), 1–33.

[10] Darshak Chhatbar, Umair Z Ahmed, and Purushottam Kar. 2020. Macer: A modular framework for accelerated compilation error repair. In *International Conference on Artificial Intelligence in Education*. Springer, 106–117.

[11] Trishul M. Chilimbi, Ben Liblit, Krishna Mehra, Aditya V. Nori, and Kapil Vaswani. 2009. HOLMES: Effective statistical debugging via efficient path profiling. In *2009 IEEE 31st International Conference on Software Engineering*.

[12] GitHub Copilot. 2022. Your AI pair programmer. https://copilot.github.com/.

[13] Paul Denny, Andrew Luxton-Reilly, and Ewan D. Tempero. 2012. All syntax errors are not equal. In *Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE '12, Haifa, Israel, July 3-5, 2012*, Tami Lapidot, Judith Gal-Ezer, Michael E. Caspersen, and Orit Hazzan (Eds.). ACM, 75–80. https://doi.org/10.1145/2325296.2325318

[14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[15] Patrick Fernandes, Miltiadis Allamanis, and Marc Brockschmidt. 2018. Structured neural summarization. *arXiv preprint arXiv:1811.01824* (2018).

[16] Cheng Fu, Huili Chen, Haolan Liu, Xinyun Chen, Yuandong Tian, Farinaz Koushanfar, and Jishen Zhao. 2019. Coda: An End-to-End Neural Program Decompiler. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. 3703–3714.

[17] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).

[18] Rahul Gupta, Aditya Kanade, and Shirish Shevade. 2019. Deep reinforcement learning for syntactic error repair in student programs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 930–937.

[19] Rahul Gupta, Soham Pal, Aditya Kanade, and Shirish Shevade. 2017. Deepfix: Fixing common c language errors by deep learning. In *Thirty-First AAAI Conference on Artificial Intelligence*.

[20] Hossein Hajipour, Apratim Bhattacharyya, Cristian-Alexandru Staicu, and Mario Fritz. 2021. SampleFix: learning to correct programs by sampling diverse fixes. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 119–133.

[21] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[22] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. CURE: Code-Aware Neural Machine Translation for Automatic Program Repair. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid, Spain, 22-30 May 2021*. IEEE, 1161–1173. https://doi.org/10.1109/ICSE43902.2021.00107

[23] Siyuan Jiang, Ameer Armaly, and Collin McMillan. 2017. Automatically generating commit messages from diffs using neural machine translation. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 135–146.

[24] Omer Katz, Yuval Olshaker, Yoav Goldberg, and Eran Yahav. 2019. Towards Neural Decompilation. *CoRR* abs/1905.08325 (2019). arXiv:1905.08325 http://arxiv.org/abs/1905.08325

[25] Jindae Kim, Jeongho Kim, Eunseok Lee, and Sunghun Kim. 2020. The effectiveness of context-based change application on automatic program repair. *Empirical Softw. Engg.* (2020).

[26] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[27] Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. 2019. Spoc: Search-based pseudocode to code. *Advances in Neural Information Processing Systems* 32 (2019).

[28] Xueyang Li, Shangqing Liu, Ruitao Feng, Guozhu Meng, Xiaofei Xie, Kai Chen, and Yang Liu. 2022. TransRepair: Context-Aware Program Repair for Compilation Errors. https://sites.google.com/view/transrepair/.

[29] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. DLFix: Context-based Code Transformation Learning for Automated Program Repair. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*.

[30] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018).

[31] Ruigang Liang, Ying Cao, Peiwei Hu, and Kai Chen. 2021. Neutron: an attention-based neural decompiler. *Cybersecurity* 4, 1 (2021), 1–13.

[32] Xiang Ling, Lingfei Wu, Saizhuo Wang, Gaoning Pan, Tengfei Ma, Fangli Xu, Alex X Liu, Chunming Wu, and Shouling Ji. 2021. Deep graph matching and searching for semantic code retrieval. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15, 5 (2021), 1–21.

[33] Shangqing Liu, Yu Chen, Xiaofei Xie, Jing Kai Siow, and Yang Liu. 2020. Retrieval-Augmented Generation for Code Summarization via Hybrid GNN. In *International Conference on Learning Representations*.

[34] Shangqing Liu, Cuiyun Gao, Sen Chen, Nie Lun Yiu, and Yang Liu. 2020. ATOM: Commit message generation based on abstract syntax tree and hybrid ranking. *IEEE Transactions on Software Engineering* (2020).

[35] Shangqing Liu, Yanzhou Li, and Yang Liu. 2022. CommitBART: A Large Pre-trained Model for GitHub Commits. *arXiv preprint arXiv:2208.08100* (2022).

[36] Shangqing Liu, Xiaofei Xie, Lei Ma, Jingkai Siow, and Yang Liu. 2021. Graphsearchnet: Enhancing gnns via capturing global dependency for semantic code search. *arXiv preprint arXiv:2111.02671* (2021).

[37] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*.

[38] Ali Mesbah, Andrew Rice, Emily Johnston, Nick Glorioso, and Edward Aftandilian. 2019. Deepdelta: learning to repair compilation errors. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 925–936.

[39] Eddie Antonio Santos, Joshua Charles Campbell, Dhvani Patel, Abram Hindle, and José Nelson Amaral. 2018. Syntax and sensibility: Using language models to detect and correct syntax errors. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 311–322.

[40] Abigail See, Peter J Liu, and Christopher D Manning. 2017. Get to the point: Summarization with pointer-generator networks. *arXiv preprint arXiv:1704.04368* (2017).

[41] Jing Kai Siow, Shangqing Liu, Xiaofei Xie, Guozhu Meng, and Yang Liu. 2022. Learning Program Semantics with Code Representations: An Empirical Study. *arXiv preprint arXiv:2203.11790* (2022).

[42] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. 2015. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems* 28 (2015).

[43] Zhensu Sun, Li Li, Yan Liu, Xiaoning Du, and Li Li. 2022. On the Importance of Building High-quality Training Datasets for Neural Code Search. *CoRR* abs/2202.06649 (2022). arXiv:2202.06649 https://arxiv.org/abs/2202.06649

[44] V. Javier Traver. 2010. On Compiler Error Messages: What They *Say* and What They *Mean*. *Adv. Hum. Comput. Interact.* 2010 (2010), 602570:1–602570:26. https://doi.org/10.1155/2010/602570

[45] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI '22: CHI Conference on Human Factors in Computing Systems, New Orleans, LA, USA, 29 April 2022 - 5 May 2022, Extended Abstracts*, Simone D. J. Barbosa, Cliff Lampe, Caroline Appert, and David A. Shamma (Eds.). ACM, 332:1–332:7.

[46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.

[47] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).

[48] Ming Wen, Junjie Chen, Rongxin Wu, Dan Hao, and Shing-Chi Cheung. 2018. Context-Aware Patch Generation for Better Automated Program Repair. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*.

[49] Bozhi Wu, Shangqing Liu, Ruitao Feng, Xiaofei Xie, Jingkai Siow, and Shang-Wei Lin. 2022. Enhancing Security Patch Identification by Capturing Structures in

Commits. *IEEE Transactions on Dependable and Secure Computing* (2022).

[50] Michihiro Yasunaga and Percy Liang. 2020. Graph-based, self-supervised program repair from diagnostic feedback. In *International Conference on Machine Learning*. PMLR, 10799–10808.

[51] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems*. 10197–10207.

[52] Yaqin Zhou, Jing Kai Siow, Chenyu Wang, Shangqing Liu, and Yang Liu. 2021. SPI: Automated Identification of Security Patches via Commits. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–27.

[53] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2019. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2019), 2224–2236.